X

(https://swayam.gov.in) (https://swayam.gov.in/nc_details/NPTEL)

pradipmaity9907@gmail.com

NPTEL (https://swayam.gov.in/explorer?ncCode=NPTEL) » The Joy of Computing using Python (course)

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Week 7 ()

Week 8 ()

Week 9 ()

Week 10 ()

Week 11 : Assignment 11

The due date for submitting this assignment has passed.

Due on 2026-04-08, 23:59 IST.

Assignment submitted on 2026-03-31, 19:59 IST

Automating Message Dispatch on WhatsApp Web Using Selenium

Background

Web automation tools are often used to reduce repetitive manual tasks. One common example is interacting with web-based messaging platforms to send reminders, notifications, or test message workflows. WhatsApp Web, being a browser-based interface, can be automated using Selenium to simulate real user actions such as clicking, typing, and pressing keys.

This case study examines a Python-based Selenium script that automates sending multiple messages to a specific contact on WhatsApp Web.

Objective

The goal of the script is to:

- Open WhatsApp Web in a browser
- Allow the user to authenticate manually via QR code
- Search for a specific contact by name
- Open the chat window of that contact
- Send the same message multiple times at fixed intervals
- Close the browser session after completion

Tools and Technologies Used

- Python
- Selenium WebDriver
- Google Chrome Browser
- ChromeDriver
- WhatsApp Web

Week 11 ()

- Browser Automation Watsapp using Python - Part 01 (unit?unit=227&lesson=228)
- Browser Automation Watsapp using Python - Part 02 (unit?unit=227&lesson=229)
- Browser Automation Watsapp using Python - Part 03 (unit?unit=227&lesson=230)
- Browser Automation Watsapp using Python - Part 04 (unit?unit=227&lesson=231)
- Fun with Calendar - Part 01 (unit?unit=227&lesson=232)
- Fun with Calendar - Part 02 (unit?unit=227&lesson=233)
- Fun with Calendar - Part 03 (unit?unit=227&lesson=234)
- Fun with Calendar - Part 04 (unit?unit=227&lesson=235)

Workflow Explanation

1. Browser Initialization
The script begins by initializing the Chrome WebDriver. This enables Python to control the Chrome browser programmatically.

Key idea:
The WebDriver acts as a bridge between Python commands and browser actions.

2. Loading WhatsApp Web
The browser navigates to the WhatsApp Web URL. Since WhatsApp requires authentication, the script pauses implicitly while the user scans the QR code using their mobile WhatsApp application.

Important observation:
Automation does not bypass authentication. Human intervention is required at this step.

3. Identifying the Target Chat
The script defines:

- A contact name
- A message string

It then constructs an XPath expression that locates a chat whose title attribute contains the contact's name. WebDriverWait is used to ensure the element is loaded before interaction
Concepts involved:

- XPath-based element selection
- Explicit waits to handle dynamic content

4. Opening the Chat
Once the contact element is located, a click action opens the chat window.

This simulates a real user selecting a chat from the chat list.

5. Locating the Message Input Box
The script finds the message input field using its class name. This element is where text messages are typed.

This step relies on understanding the DOM structure of WhatsApp Web.

6. Sending Messages in a Loop
A loop sends the same message multiple times:

- The message text is typed
- The Enter key is pressed to send
- A delay is added between messages

Why the delay matters:

- Prevents rapid-fire actions
- Reduces the risk of detection or UI failure
- Mimics human behavior

7. Closing the Session
After sending all messages, the browser is closed gracefully using the quit method.

Fun with Calendar - Part 05 (unit?unit=227&lesson=236)
Fun with Calendar - Part 06 (unit?unit=227&lesson=237)
Fun with Calendar - Part 07 (unit?unit=227&lesson=238)
Fun with Calendar - Part 08 (unit?unit=227&lesson=239)
Fun with Calendar - Part 09 (unit?unit=227&lesson=240)
Fun with Calendar - Part 10 (unit?unit=227&lesson=241)
Fun with Calendar - Part 11 (unit?unit=227&lesson=242)
Fun with Calendar - Part 12 (unit?unit=227&lesson=243)
Quiz: Week 11 : Assignment 11 (assessment?name=661)
Week 11: Programming Assignment 1 (/noc26_cs84/

This ensures system resources are released properly.

```
from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as ec
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome('path_to_chromedriver.exe')

driver.get("https://web.whatsapp.com/")

recipient_name = "Ravi Kiran"
message_text = "Message sent using Python"

xpath = f"//span[contains(@title, '{recipient_name}')]"
target = WebDriverWait(driver, 10).until(ec.presence_of_element_located((By.XPATH, xpath)))
target.click()

input_box = driver.find_element(By.CLASS_NAME, "_1Plpp")

for i in range(50):
    input_box.send_keys(message_text)
    input_box.send_keys(Keys.RETURN)
    time.sleep(1)

driver.quit()
```

1) Why is WebDriverWait preferred over using a fixed time.sleep() before locating the chat element? ***1 point***

- It waits only until the required element becomes available, making the script more reliable
- It reduces CPU usage of the Python program
- It prevents WhatsApp from detecting automation
- It speeds up QR code scanning

No, the answer is incorrect.
Score: 0

Accepted Answers:
It waits only until the required element becomes available, making the script more reliable

2) What is the main limitation of using the contact's **name** in the XPath selector? ***1 point***

- It works only for group chats
- It may fail if multiple contacts share similar names
- It cannot locate encrypted chats
- It increases page load time

Yes, the answer is correct.
Score: 1

Accepted Answers:
It may fail if multiple contacts share similar names

3) Why does the script require **manual QR code scanning** instead of automating login? ***1 point***

progassignment?name=647)

Week 11: Programming Assignment 2 (/noc26_cs84/progassignment?name=648)

Week 11 : Programming Assignment 3 (/noc26_cs84/progassignment?name=649)

Week 11 Feedback Form: The Joy of Computing using Python (unit?unit=227&lesson=244)

Week 12 ()

Text Transcripts ()

Download Videos ()

- Selenium does not support camera access
- WhatsApp Web blocks JavaScript execution
- Authentication is tied to a real user session for security
- ChromeDriver cannot load HTTPS websites

Yes, the answer is correct.
Score: 1

Accepted Answers:
Authentication is tied to a real user session for security

4) What would most likely happen if the delay between messages (time.sleep(1)) were removed? ***1 point***

- Messages would not be sent at all
- The browser would crash immediately
- Selenium would raise a syntax error
- The script could trigger UI failures or platform restrictions

No, the answer is incorrect.
Score: 0

Accepted Answers:
The script could trigger UI failures or platform restrictions

5) Which Selenium feature is primarily responsible for simulating a **real user pressing Enter**? ***1 point***

- Keys.RETURN
- By.CLASS_NAME
- driver.get()
- WebDriverWait

Yes, the answer is correct.
Score: 1

Accepted Answers:
Keys.RETURN

6) Why is automation of messaging platforms considered ethically sensitive? ***1 point***

- It consumes more internet bandwidth
- It can lead to misuse such as spamming or harassment
- It requires paid browser drivers
- It increases system memory usage

No, the answer is incorrect.
Score: 0

Accepted Answers:
It can lead to misuse such as spamming or harassment

7) Which design choice makes this script **platform-dependent**? ***1 point***

- Use of Python language
- Reliance on internet connectivity
- Dependence on WhatsApp Web's DOM structure

- (selected) Use of loops for repetition

No, the answer is incorrect.
Score: 0

Accepted Answers:
Dependence on WhatsApp Web's DOM structure

8) Why is driver.quit() preferred over simply closing the browser window manually? ***1 point***

- It saves execution time
- It sends logout signals to WhatsApp
- It keeps the session active for reuse
- (selected) It properly terminates the WebDriver and releases system resources

Yes, the answer is correct.
Score: 1

Accepted Answers:
It properly terminates the WebDriver and releases system resources

Case Study: Automated Academic Schedule Analyzer

Background

A university administration system needs to **analyze academic schedules** for a semester. The system should be able to:

- Track important academic dates (semester start, exams, holidays)
- Determine weekdays for given dates
- Identify leap years (important for February schedules)
- Generate monthly calendars for planning purposes
- Compute gaps between important academic events

To implement this functionality, the developers decide to use Python's built-in **datetime** and **calendar** modules.

Part 1: Working with Dates and Time (datetime Module)

The system begins by capturing the **current date and time** when the administrator logs in.

```
import datetime

login_time = datetime.datetime.now()
print(login_time)
```

This timestamp is later used for logging administrative actions.

Creating Fixed Academic Dates

The semester officially starts on **August 5, 2025**, and the mid-semester exam is scheduled for **October 12, 2025**.

```
semester_start = datetime.date(2025, 8, 5)
mid_exam = datetime.date(2025, 10, 12)
```

Each of these objects stores only the date, not the time.

Calculating Duration Between Events

The system calculates how many days students have between the semester start and the mid-semester exam.

```
gap = mid_exam - semester_start
print(gap.days)
```

This value is later used to ensure syllabus coverage is realistic.

Extracting Date Components

For reporting purposes, individual components of a date are extracted:

```
year = semester_start.year
month = semester_start.month
day = semester_start.day
```

These values are stored separately in the database.

Formatting Dates for Reports

To generate human-readable academic notices, dates are formatted:

```
formatted_date = semester_start.strftime("%A, %d %B %Y")
print(formatted_date)
```

Part 2: Academic Calendar Analysis (calendar Module)

To assist with planning lectures and exams, the system generates a **monthly calendar**.

```
import calendar

print(calendar.month(2025, 8))
```

This helps administrators visualize holidays and weekends.

Checking Leap Year Constraints

Since February schedules depend on leap years, the system verifies whether **2024** is a leap year.

```
calendar.isleap(2024)
```

This check prevents incorrect date assumptions during scheduling.

Identifying Weekdays for Exams

The day of the week for the mid-semester exam is computed:

```
exam_day = calendar.weekday(2025, 10, 12)
print(calendar.day_name[exam_day])
```

This ensures exams are not accidentally scheduled on weekends.

Counting Days in a Month

To calculate the total number of lecture days in a month:

```
days_in_august = calendar.monthrange(2025, 8)[1]
print(days_in_august)
```

This value is later filtered to exclude weekends and holidays.

9) The system computes the gap between semester_start and mid_exam using: ***1 point***

```
gap = mid_exam - semester_start
print(gap.days)
```

What does gap.days logically represent?

- The number of calendar days between the two academic events
- The number of weekdays between the two dates
- The number of lecture days excluding holidays
- The number of months between the two dates

No, the answer is incorrect.
Score: 0

Accepted Answers:
The number of calendar days between the two academic events

10) If semester_start is created as: ***1 point***

```
semester_start = datetime.date(2025, 8, 5)
```

Which of the following attributes is **NOT directly available** from this object?

- semester_start.year
- semester_start.hour
- semester_start.month
- semester_start.day

No, the answer is incorrect.
Score: 0

Accepted Answers:
semester_start.hour

11) The following code is used to format a date for notices: ***1 point***

```
semester_start.strftime("%A, %d %B %Y")
```

Which part of the output is produced specifically by %A?

- Numeric day of the month
- Full name of the month
- Full name of the weekday
- Four-digit year

No, the answer is incorrect.
Score: 0

Accepted Answers:
Full name of the weekday

12) Why does the system check the following condition? ***1 point***

```
calendar.isleap(2024)
```

- To determine whether weekends change in 2024

- To confirm February has 29 days in that year
- To find how many weekdays exist in 2024
- To calculate the total number of academic days

No, the answer is incorrect.
Score: 0

Accepted Answers:
To confirm February has 29 days in that year

13) The system determines the exam weekday using: ***1 point***

```
exam_day = calendar.weekday(2025, 10, 12)
calendar.day_name[exam_day]
```

Why is this a two-step process?

- calendar.weekday() returns a string index
- calendar.weekday() returns a month name
- calendar.weekday() returns an integer mapped to a weekday
- calendar.day_name only works for leap years

No, the answer is incorrect.
Score: 0

Accepted Answers:
calendar.weekday() returns an integer mapped to a weekday

14) Consider the code: ***1 point***

```
days_in_august = calendar.monthrange(2025, 8)[1]
```

What does the index [1] specifically retrieve?

- The weekday of the first day of August
- The number of Sundays in August
- The total number of days in August
- The number of weekdays in August

No, the answer is incorrect.
Score: 0

Accepted Answers:
The total number of days in August

15) Why is datetime.datetime.now() used instead of datetime.date.today() for login tracking? ***1 point***

- date.today() is slower
- Login tracking requires both date and time
- datetime.date cannot be stored in logs
- datetime.now() works only on servers

No, the answer is incorrect.
Score: 0

Accepted Answers:
Login tracking requires both date and time